Hoodly

Justification de l'Architecture et des Technologies

1. Introduction

Ce document explique les choix d'architecture technique de Hoodly. On ne voulait pas se contenter de lister les technos utilisées, mais plutôt expliquer pourquoi on a fait tel ou tel choix, ce qu'on a comparé, et ce que ça nous apporte concrètement.

Tous les choix qu'on a faits s'appuient sur une logique simple : aller chercher le meilleur compromis entre performance, maintenabilité et respect du sujet. Le projet est imposant (une dizaine de fonctionnalités majeurs, trois interfaces, deux bases de données), donc il fallait que l'architecture tienne dans la durée sans devenir ingérable.

2. Framework backend : NestJS

Le sujet impose Node.js avec Express. NestJS utilise Express.js comme serveur HTTP sous-jacent, donc techniquement on est bien sur Express. La différence, c'est que NestJS ajoute une couche d'abstraction au-dessus qui structure le projet et simplifie le développement.

On a comparé Express pur et NestJS. Avec Express, au début c'est rapide : quelques lignes et un serveur tourne. Mais sur un projet avec dix modules différents (utilisateurs, zones, incidents, services, événements, messages, votes, documents), ça devient vite le chaos. Pas de structure imposée, pas de système de modules, pas d'injection de dépendances native. On se retrouve à tout organiser soi-même et à réécrire les mêmes patterns partout.

NestJS, c'est l'inverse. La structure est imposée : modules, controllers, services, DTOs. Chaque fonctionnalité a son propre module avec tout ce dont elle a besoin. L'injection de dépendances est native grâce au conteneur IoC, les guards d'authentification sont prêts à l'emploi, et la documentation Swagger se génère automatiquement à partir des décorateurs. Pour un projet de cette envergure, ça fait gagner un temps considérable et ça garantit une cohérence dans le code.

Le TypeScript est natif chez NestJS, pas ajouté après coup. On a du typage fort partout, ce qui évite une bonne partie des bugs en développement.

En résumé : NestJS est un superset d'Express. Le code tourne sur Express, respecte l'esprit du sujet, mais on bénéficie d'une architecture structurée et maintenable. Pour un projet avec autant de modules, c'était le choix

logique.

3. Architecture applicative

Pour organiser le code du backend, on a opté pour une architecture en **feature modules**. L'idée, c'est que chaque fonctionnalité métier de la plateforme ait son propre module autonome : *users/, zones/, incidents/, services/, events/, messages/, votes/, documents/*. Chaque module contient tout ce qui le concerne : le controller qui gère les routes, le service qui contient la logique métier, les DTOs qui définissent la forme des requêtes et réponses, et le schéma Mongoose pour la base de données.

Concrètement, si on veut travailler sur les zones, tout est au même endroit dans *modules/zones/*. Pas besoin de chercher dans quatre dossiers différents, tout est regroupé par domaine.

Au-dessus des modules métier, on a deux dossiers transversaux. Le dossier *core/* contient les fonctionnalités critiques utilisées par tout le projet, notamment l'authentification. Le dossier *shared/* contient des utilitaires réutilisables comme les pipes de validation ou les helpers. C'est là qu'on met les éléments qui ne sont pas spécifiques à un module mais utiles à plusieurs.

Cette organisation nous apporte trois choses importantes. D'abord, le découplage : les modules ne dépendent pas directement les uns des autres. Le module incidents importe users uniquement pour le type UserRole, il ne touche pas à la logique interne du module users. Ensuite, la scalabilité : ajouter un nouveau module consiste à créer un dossier *modules/nouveau-module/* avec la même structure que les autres, et ajouter une ligne d'import dans *app.module.ts*. Aucun fichier existant n'est modifié. Enfin, la lisibilité : un développeur qui rejoint le projet peut comprendre l'organisation en quelques minutes.

On a envisagé d'autres architectures. Une structure à plat avec tout en vrac dans *src/* aurait été simple au début mais ingérable avec dix modules. Une architecture hexagonale (ports et adapters) serait trop complexe pour un projet étudiant avec un seul protocole d'entrée (HTTP REST). Notre approche est le bon juste milieu.

4. Base de données : MongoDB

Pour la base de données principale, on a choisi MongoDB (NoSQL) plutôt qu'un SQL classique comme PostgreSQL.

La raison principale, c'est la flexibilité des schémas. Sur Hoodly, les structures de données varient énormément d'un module à l'autre : un utilisateur n'a rien à voir avec un message, un événement n'a rien à voir avec un contrat. Avec du SQL, chaque modification de schéma implique des migrations, ce qui est lourd sur un projet en développement où les besoins évoluent vite. Avec MongoDB, on peut ajouter un champ à un document sans toucher aux autres, le schéma est souple.

La deuxième raison, c'est le support natif de GeoJSON. Les zones de quartier sont des polygones géographiques, et MongoDB supporte ce format directement avec des opérations géospatiales intégrées (tester si un point est dans un polygone, chercher les zones à proximité, etc.). En SQL, il faudrait installer PostGIS comme extension supplémentaire, ce qui rajoute de la complexité.

La troisième raison, c'est la nature même de certaines données. Les messages contiennent des médias imbriqués, les événements ont des listes de participants, les votes ont des tableaux d'options. Ce sont des structures JSON naturelles qu'on stocke directement comme des documents, sans avoir à les éclater en plusieurs tables avec des jointures.

Pour l'interaction avec MongoDB, on utilise Mongoose comme ODM. C'est le standard dans l'écosystème NestJS. Mongoose nous apporte la validation des schémas côté application (avant même d'arriver en base), les hooks (actions à exécuter avant ou après une sauvegarde), et le typage TypeScript via le package **@nestjs/mongoose.** C'est comme un ORM mais adapté à MongoDB.

5. Base de donnée graphe : Neo4j

On a besoin d'une deuxième base de données, et c'est Neo4j. Son rôle est complètement différent de MongoDB, Neo4j ne stocke pas les données métier, elle gère uniquement les relations entre utilisateurs pour alimenter le moteur de recommandations.

Pourquoi une base à part ? Parce que les recommandations qu'on veut faire reposent sur des traversées de graphe, trouver les voisins les plus connectés, repérer les événements populaires dans un cercle d'amis, identifier des services compatibles. MongoDB peut stocker des relations, mais quand il faut traverser trois ou quatre niveaux de connexions (ami d'un ami qui a participé à un événement que mon autre ami a liké), les requêtes deviennent vite lourdes et lentes. Neo4j est conçu exactement pour ça, c'est son métier de base.

Le fonctionnement est le suivant : chaque utilisateur est un nœud dans le graphe. Les relations entre nœuds représentent les interactions : HELPED (a aidé), ATTENDED (a participé à), INTERESTED_IN (s'intéresse à). Quand un voisin effectue un service pour un autre dans MongoDB, on crée la relation correspondante dans Neo4j. Pareil quand un utilisateur participe à un événement ou montre de l'intérêt pour un type de service.

Les requêtes se font en Cypher, le langage natif de Neo4j. Par exemple, pour trouver les voisins fiables d'un utilisateur, on peut écrire une requête qui traverse le graphe en suivant les relations HELPED, pondère les résultats par le nombre d'interactions et renvoie les voisins les plus actifs. C'est quelque chose qui prendrait des jointures complexes en SQL et des agrégations lourdes en MongoDB.

Le backend fait les appels aux deux bases en parallèle : MongoDB pour les données classiques, Neo4j pour les suggestions. Les deux se complètent sans se chevaucher.

6. Authentification : Auth0 + JWT

L'authentification est un sujet critique, surtout sur une plateforme où les utilisateurs signent des documents et échangent des services. On a fait le choix d'utiliser Auth0 plutôt que de coder notre propre système, et d'utiliser JWT comme mécanisme de tokens.

Pourquoi Auth0 ?

La principale raison, c'est que le sujet impose le SSO entre l'application web et l'application Java Desktop. Mettre en place un SSO soi-même entre deux plateformes différentes, c'est un vrai casse-tête. Auth0 le gère nativement : un seul compte, deux interfaces. L'admin se connecte une fois et a accès au back-office web et au client Java sans devoir ressaisir ses identifiants.

Le MFA est aussi imposé par le sujet pour les actions sensibles. Auth0 fournit le multi-facteurs prêt à l'emploi, pas besoin de coder nous-mêmes la gestion des clés secrètes, les QR codes, la vérification des codes, tout est géré par le service.

Pour l'application bureau Java, Auth0 supporte le flux PKCE (Proof Key for Code Exchange), qui est la méthode recommandée pour les applications desktop. Contrairement à un flux classique avec un secret client, le PKCE utilise des code challenge/code verifier temporaires, donc aucun secret à stocker dans l'appli. C'est plus sécurisé.

Côté gestion des rôles, on les stocke dans les claims du JWT. Quand Auth0 génère le token, il inclut le rôle de l'utilisateur dans le payload. Le backend n'a donc pas besoin de faire une requête en base à chaque appel pour vérifier les droits, il lit directement le rôle depuis le token !

Le flux complet fonctionne comme suit. L'utilisateur se connecte via Auth0, reçoit un JWT, et le transmet dans le header Authorization: Bearer <token> à chaque requête. Le backend vérifie le token grâce aux clés publiques JWKS d'Auth0 (un Guard NestJS s'en occupe automatiquement). Il vérifie la signature, l'expiration, l'émetteur et l'audience. Si tout est bon, le payload est injecté dans la requête et le traitement continue. Sinon, c'est un 401.

À chaque première connexion, le backend synchronise le profil Auth0 avec MongoDB : création si le compte n'existe pas encore, mise à jour des infos si elles ont changé. Comme ça on a notre propre copie des données utilisateur et on n'est pas dépendant d'Auth0 pour les fonctionnalités métier.

7. Frontend : React

Pour les interfaces web (l'app habitant et le back-office admin), on a choisi React. C'est le framework imposé par le sujet, mais c'est aussi le choix le plus logique dans l'écosystème actuel : communauté énorme, écosystème de librairies mature, et beaucoup de devs le connaissent.

Côté build, on utilise Vite plutôt que Create React App. C'est plus rapide en développement grâce au Hot Module Replacement quasi instantané, et la configuration est plus légère. Pour un projet avec deux frontends à maintenir, la rapidité de build fait une vraie différence au quotidien.

Pour la gestion des données côté serveur, on utilise TanStack Query (anciennement React Query). C'est une librairie qui gère le fetching, le cache et la synchronisation des données. Quand on appelle une API, TanStack Query met les résultats en cache, les rafraîchit automatiquement en arrière-plan, et gère les états de loading et d'erreur. Ça évite d'écrire de la logique de cache manuelle partout dans l'appli.

Pour l'état global, on utilise Zustand. C'est un state manager léger, beaucoup plus simple que Redux. On n'a pas besoin d'un outil aussi lourd que Redux pour notre cas : Zustand fait la même chose avec moins de boilerplate. On l'utilise pour les états partagés entre composants comme l'utilisateur connecté, les filtres actifs ou les préférences d'affichage.

Pour les appels HTTP, c'est Axios. C'est un client HTTP classique qui permet d'intercepter les requêtes et réponses, utile pour injecter automatiquement le token JWT dans les headers et gérer les erreurs globalement.

Côté UI, on a fait un choix pragmatique. Tailwind CSS pour le styling utilitaire, Radix UI pour les composants accessibles (dialogs, selects, modales) et ShadcnUI qui est une couche de styles par-dessus Radix. L'avantage, c'est qu'on a des composants accessibles et personnalisables sans dépendre d'un design system lourd comme Material UI. Lucide React nous fournit nos icônes.

8. Application Java Desktop

L'application desktop est un cas un peu à part dans le projet. C'est une application Java avec JavaFX, réservée à l'administrateur, qui gère les incidents et les statistiques du quartier.

Pourquoi Java et JavaFX ?

C'est imposé par le sujet, mais ça fait sens. L'idée est d'avoir un client lourd qui fonctionne indépendamment du navigateur, avec une interface native et le mode offline-first. JavaFX fournit les composants UI classiques (tableaux, formulaires, graphiques) et permet de styliser l'interface avec du CSS, ce qui facilite la personnalisation.

Pour les appels au backend, on utilise OkHttp comme client HTTP. C'est le standard dans l'écosystème Java, léger et performant. Les données JSON sont traitées avec Jackson pour la sérialisation/désérialisation. Maven gère les dépendances et le build.

Pourquoi le mode offline-first ?

L'admin n'est pas toujours connecté. Il peut être dans une zone sans wifi, dans les transports, ou avoir une coupure internet. Sans mode offline, l'application serait inutilisable dans ces cas-là. Le sujet le demande explicitement, et c'est logique pour un outil de gestion d'incidents : un incident urgent ne peut pas attendre qu'Internet revienne.

Pour ça, on utilise SQLite comme base locale. C'est une base légère, embarquée directement dans le .jar, pas besoin d'installer un serveur de base de données sur la machine de l'admin. Au premier lancement, l'appli récupère les données depuis le backend et les stocke dans SQLite. Ensuite, en mode déconnecté, tout passe par la base locale.

Le plus délicat, c'est la synchronisation. Quand la connexion revient, l'appli envoie les modifications locales au backend. On gère les conflits avec une règle simple : le dernier modifié gagne. Si un incident a été modifié à la fois offline et online, la version serveur est prioritaire, et l'admin est notifié du conflit. C'est pas le système le plus sophistiqué, mais c'est robuste et prévisible.

9. Validation des données

Sur un projet comme Hoodly où les utilisateurs envoient des données via des formulaires, des uploads et des requêtes API, la validation est cruciale. On a mis en place un système à deux niveaux pour éviter que des données corrompues ou malformées arrivent jusqu'à la base.

Le premier niveau, c'est la validation sur les DTOs (Data Transfer Objects) avec **class-validator**. Chaque DTO définit les règles de validation déclarativement : ce champ est une chaîne, ce champ ne doit pas être vide, ce champ doit être un email valide, etc. C'est NestJS qui s'occupe d'exécuter ces validations automatiquement avant que la requête n'atteigne le controller. Si une validation échoue, le client reçoit une erreur 400 avec les détails de ce qui ne va pas. L'avantage, c'est que c'est lisible et maintenable : on voit directement les règles en regardant le DTO, pas besoin de chercher dans la logique du controller.

Le deuxième niveau, c'est un pipe personnalisé **MongoIdValidationPipe** qui vérifie que les identifiants passés dans les routes sont des ObjectIds MongoDB valides. C'est un cas classique : un client envoie */api/users/abc123* et sans cette vérification, MongoDB renverrait une erreur cryptique. Notre pipe intercepte l'ID avant, vérifie qu'il a le bon format, et renvoie une erreur claire si ce n'est pas le cas.

Ces deux niveaux fonctionnent ensemble. Les DTOs valident le contenu des requêtes (le body, les paramètres de query), le pipe valide les paramètres de route. Rien ne passe sans être vérifié.

On utilise aussi *class-transformer* pour convertir les objets bruts reçus du client en instances de classes typées. C'est indispensable pour que les décorateurs de *class-validator* fonctionnent correctement.

9. Validation des données

Sur un projet comme Hoodly où les utilisateurs envoient des données via des formulaires, des uploads et des requêtes API, la validation est cruciale. On a mis en place un système à deux niveaux pour éviter que des données corrompues ou malformées arrivent jusqu'à la base.

Le premier niveau, c'est la validation sur les DTOs (Data Transfer Objects) avec **class-validator**. Chaque DTO définit les règles de validation déclarativement : ce champ est une chaîne, ce champ ne doit pas être vide, ce champ doit être un email valide, etc. C'est NestJS qui s'occupe d'exécuter ces validations automatiquement avant que la requête n'atteigne le controller. Si une validation échoue, le client reçoit une erreur 400 avec les détails de ce qui ne va pas. L'avantage, c'est que c'est lisible et maintenable : on voit directement les règles en regardant le DTO, pas besoin de chercher dans la logique du controller.

Le deuxième niveau, c'est un pipe personnalisé **MongoIdValidationPipe** qui vérifie que les identifiants passés dans les routes sont des ObjectIds MongoDB valides. C'est un cas classique : un client envoie */api/users/abc123* et sans cette vérification, MongoDB renverrait une erreur cryptique. Notre pipe intercepte l'ID avant, vérifie qu'il a le bon format, et renvoie une erreur claire si ce n'est pas le cas.

Ces deux niveaux fonctionnent ensemble. Les DTOs valident le contenu des requêtes (le body, les paramètres de query), le pipe valide les paramètres de route. Rien ne passe sans être vérifié.

On utilise aussi *class-transformer* pour convertir les objets bruts reçus du client en instances de classes typées. C'est indispensable pour que les décorateurs de *class-validator* fonctionnent correctement.

10. Conteneurisation et déploiement

Le sujet demande que les différentes applications soient conteneurisées. On utilise Docker pour ça.

Le principe est simple : chaque service tourne dans son propre conteneur isolé. Un conteneur pour le backend NestJS, un pour MongoDB, un pour Neo4j, un pour le frontend React. Docker Compose orchestre le tout avec un seul fichier de configuration. Un docker-compose up et tout le stack est lancé, que ce soit en local, en test ou en production.

L'avantage de l'architecture en modules qu'on a choisie, c'est que la conteneurisation est naturelle. Chaque module est autonome, donc chaque conteneur l'est aussi. Pas de dépendances cachées entre les services.

On prévoit de séparer les environnements : développement et production. Chaque environnement a ses propres variables d'environnement (les URLs des bases de données, les clés Auth0, les secrets JWT). Les secrets ne sont jamais codés en dur dans le code, ils passent par les variables d'environnement de Docker ou les secrets managers.

Pour le déploiement, on vise Railway comme plateforme cloud. C'est simple à configurer, ça supporte Docker nativement, et ça permet de déployer rapidement sans gérer une infrastructure complexe.

Le sujet demande aussi des tests : unitaires, d'intégration, E2E. On utilise Jest pour les tests unitaires et d'intégration côté backend, et on prévoit des tests E2E pour vérifier les parcours utilisateurs complets. Les tests s'intègrent dans le pipeline de déploiement pour garantir que rien de cassé ne passe en production.

11. Récapitulatif de nos choix techniques

Composant	Choix	Justification
Runtime	Node.js 22	Imposé par le sujet
Framework backend	NestJS 11	Structure, injection de dépendances, TypeScript natif, au-dessus d'Express
Base principale	MongoDB	Imposé + GeoJSON natif + schéma flexible + support natif des documents JSON
Base recommandations	Neo4j	Traversée de graphe performante, spécialisé dans les relations entre nœuds
Base locale Java	SQLite	Légère, embarquée dans le .jar, pas besoin de serveur
ODM	Mongoose	Standard NestJS, validation des schémas, hooks, typage TypeScript
Authentification	Auth0 + JWT	SSO web/Java, MFA prêt à l'emploi, PKCE pour desktop, JWT stateless
Validation	class-validator + pipe custom	Double couche, déclaratif, intégré à NestJS
Frontend	React 19 + Vite	Imposé, écosystème mature, Vite pour la rapidité de build
State management	Zustand + TanStack Query	Zustand pour l'état global léger, TanStack pour le fetching/cache
UI	Tailwind + Radix + ShadcnUI	Accessible, personnalisable, pas de design system lourd
Map API	**Mapbox GL JS**	Carte interactive pour l'affichage et le dessin des polygones de

		quartiers. C'est un service externe avec une API JavaScript
HTTP client	Axios	Interception des requêtes, injection automatique du JWT
Temps réel	Socket.io	WebSocket natif, temps réel pour la messagerie et les statuts
Desktop	Java + JavaFX	Imposé, UI native, support CSS
Client HTTP Java	OkHttp	Standard Java, léger et performant
Sérialisation	Jackson	**Librairie de sérialisation/désérialisation JSON. Quand le backend renvoie du JSON, Jackson le convertit en objets Java et inversement.**
Documentation API	Swagger/OpenAPI	Auto-généré par NestJS, imposé par le sujet
Conteneurisation	Docker + Docker Compose	Isolation des services, orchestration simple, imposé par le sujet
Déploiement	Railway	Simple, support Docker, adapté au projet étudiant
Tests	Jest (unitaire/intégration)	Intégré à NestJS, standard Node.js
Langage	TypeScript (web) / Java (desktop)	Typage fort des deux côtés
Packages	pnpm	Rapide, efficace, économise l'espace disque